

Assignment-20.5

Name: Navya Revuri

HTNO: 2303A52483

Batch-44

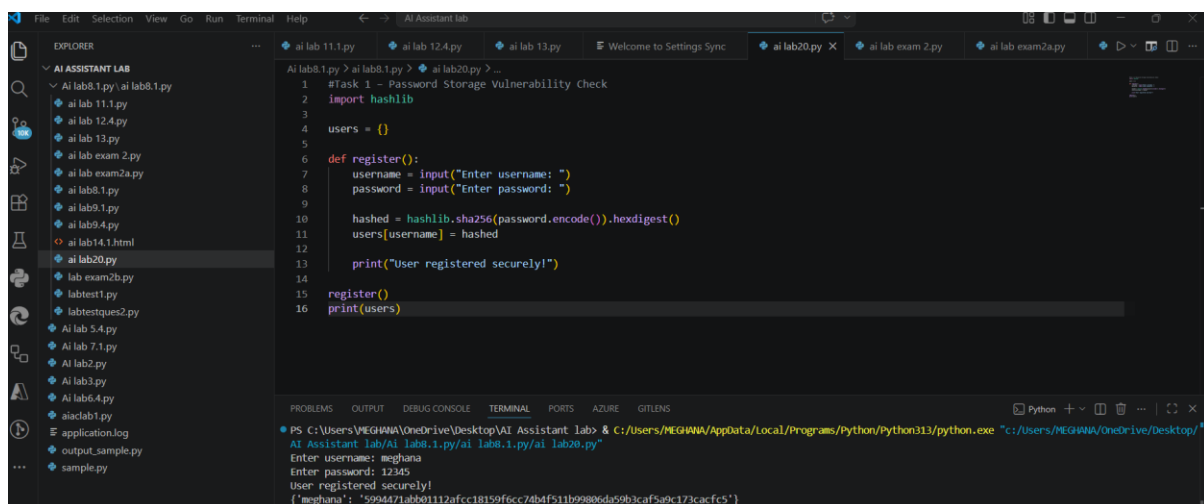
Task 1 – Password Storage Vulnerability Check

Prompt: #Generate a Python program for user registration.

Store username and password in a file.

Allow multiple users to register.

Display stored data after registration.



The screenshot shows a VS Code editor with a file explorer on the left containing various Python files. The main editor displays a Python script named 'lab20.py' with the following code:

```
1 #Task 1 - Password Storage Vulnerability Check
2 import hashlib
3
4 users = {}
5
6 def register():
7     username = input("Enter username: ")
8     password = input("Enter password: ")
9
10    hashed = hashlib.sha256(password.encode()).hexdigest()
11    users[username] = hashed
12
13    print("User registered securely!")
14
15 register()
16 print(users)
```

The terminal at the bottom shows the execution of the script:

```
PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant Lab> & C:\Users\MEGHANA\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\MEGHANA\OneDrive\Desktop\AI Assistant Lab\AI Lab\lab20.py"
Enter username: meghana
Enter password: 12345
User registered securely!
{'meghana': '5994471abb01112afcc18159f6cc74b4f511b99806da50b3caf5a0c173cacfc5'}
```

Observation: The generated code stores passwords in plain text format.

This means anyone accessing the file can read user passwords directly.

It does not use any encryption or hashing method.

This creates a major security risk and violates best practices.

Attackers can easily misuse user credentials.

Passwords should always be stored using secure hashing.

Task 2 – Hardcoded Secrets Detection

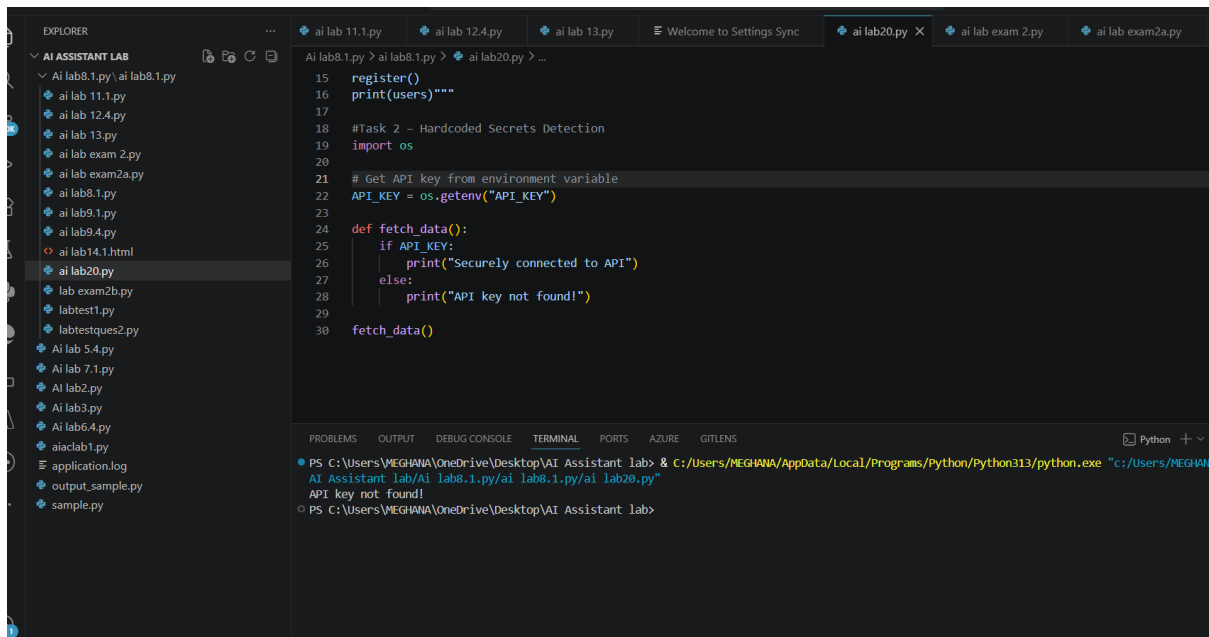
Prompt:

A Python script to connect to an external API.

Include an API key in the code.

Fetch and display data from the API.

Use simple requests library



Observation:

The API key is directly written inside the code.

This exposes sensitive credentials if the code is shared publicly.

Anyone can reuse or misuse the API key.

It is not stored securely or separately.

Best practice is to use environment variables.

Secrets should never be hardcoded in source files.

Task 3 – File Upload Security Validation

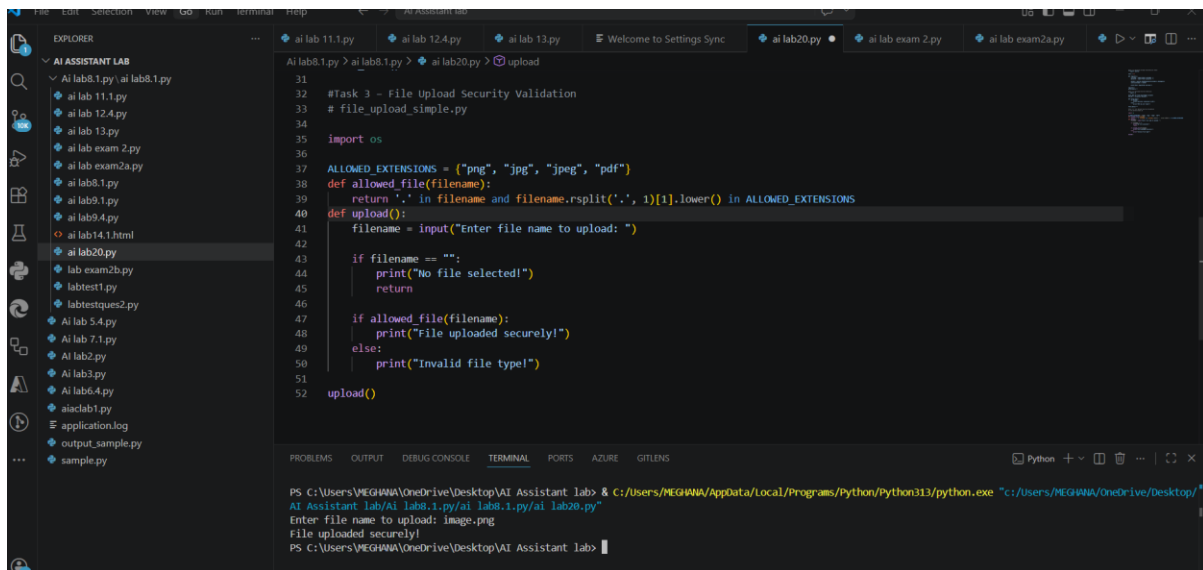
Prompt:

Generate a Flask application for file upload.

Allow users to upload files through a form.

Save uploaded files to a folder.

Display upload success message.



```
31
32 #Task 3 - File Upload Security Validation
33 # file_upload_simple.py
34
35 import os
36
37 ALLOWED_EXTENSIONS = ("png", "jpg", "jpeg", "pdf")
38 def allowed_file(filename):
39     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
40
41 def upload():
42     filename = input("Enter file name to upload: ")
43
44     if filename == "":
45         print("No file selected!")
46         return
47
48     if allowed_file(filename):
49         print("File uploaded securely!")
50     else:
51         print("Invalid file type!")
52
53 upload()
```

PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant Lab> & C:\Users\MEGHANA\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\MEGHANA\OneDrive\Desktop\AI Assistant Lab\AI lab8.1.py\ai lab8.1.py\ai lab20.py"

Enter file name to upload: image.png

File uploaded securely!

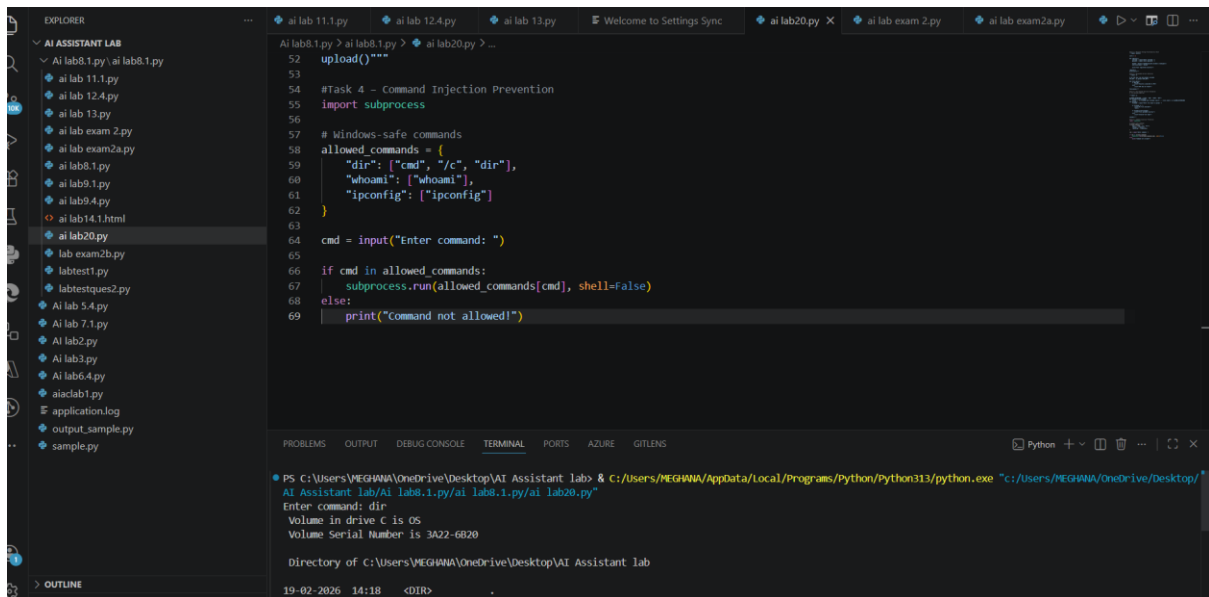
PS C:\Users\MEGHANA\OneDrive\Desktop\AI Assistant Lab>

Observation: The code allows uploading any type of file without restriction. There is no validation for file type or size. Malicious files like .exe or scripts can be uploaded. This can lead to server compromise or malware execution. Filename is not sanitized, causing path traversal risk. Secure validation and filtering are required.

Task 4 – Command Injection Prevention

Prompt:

- # Generate a Python script that executes system commands.
- # Take command input from the user.
- # Run the command using operating system
- # Display the output.



Observation:

The script directly executes user input as a system command.

This creates a serious command injection vulnerability.

Users can run harmful commands like deleting files.

There is no validation or restriction on input.

Using operating system is unsafe in this context.

Safer methods like subprocess with validation should be used.

Task 5 – Secure Input Handling in Web Forms

Prompt:

Generate an HTML form with Flask backend.

```
# Take user name and feedback as input.
```

```
# Display submitted data on a new page.
```

Use simple form handling

